

Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

4. Listas

Listas y funciones sobre listas

```
sumatorio :: Num a => [a] -> a
sumatorio [] = 0
sumatorio (x:xs) = x + sumatorio xs
```

- `[a]` representa una lista con elementos de un cierto tipo, al que llamamos `a`.
 - Todos los elementos de una lista han de ser del mismo tipo...
 - * al contrario que en las tuplas.
- `[]` es la lista vacía.
- El patrón `x:xs` representa una lista cuyo primer elemento es `x` y cuyo resto es `xs`.
 - Necesitamos que esté entre paréntesis, si no, `sumatorio x` tendría prioridad.
- Esta función `sumatorio` está predefinida en Haskell como `sum`.
- Es común usar nombres *en plural* para listas, como `xs`.
- `[1, 2, 3] = 1:[2, 3] = 1:2:[3] = 1:2:3:[]`.
- También en patrones:

```
sumaRara :: Num a => [a] -> a
sumaRara [] = 0
sumaRara [x] = x
sumaRara (x1:x2:xs) = x1 + x2 + sumaRara xs
```

- Otro ejemplo:

```
longitud :: [a] -> Int
longitud [] = 0
longitud (_:xs) = 1 + longitud xs
```

- Esta función está predefinida como `length`.

```
cabeza :: [a] -> a
cabeza (x:_) = x
```

- Esta función está predefinida como `head`.
- `head []` (y `cabeza []`) produce un error.

```

alReves :: [a] -> [a]
alReves [] = []
alReves (x:xs) = alReves xs ++ [x]

```

- Esta función está predefinida como `reverse`.
- Añadir un elemento al inicio de una lista es fácil: `x:xs`.
- Para añadirlo al final: `xs ++ [x]`.
 - Es más eficiente añadir al principio.
 - En otros lenguajes suele ser más eficiente añadir al final.

```

coger :: Int -> [a] -> [a]
coger 0 _ = []
coger _ [] = []
coger n (x:xs) = x : coger (n-1) xs

```

- Esta función está predefinida como `take`.
- Muchas funciones sobre listas son recursivas.
- `type String = [Char]`
 - `"abc" = ['a', 'b', 'c'] = 'a':'b':'c':[]`

Algunas funciones predefinidas sobre listas

- <https://hackage.haskell.org/package/base/docs/Prelude.html>
- `[1, 2] ++ [3] ~> [1, 2, 3]` (concatenación)
- `[1, 2, 3] !! 1 ~> 2` (índice basado en 0; tiene coste lineal en el índice)
- `head [1, 2, 3] ~> 1`
- `tail [1, 2, 3] ~> [2, 3]`
- `last [1, 2, 3] ~> 3`
- `init [1, 2, 3] ~> [1, 2]` (todo menos el último elemento)
- `length [1, 2, 3] ~> 3`
- `null [1, 2, 3] ~> False` (¿Es la lista vacía?)
 - `null xs` es mejor que `xs == []`.
- `reverse [1, 2, 3] ~> [3, 2, 1]`
- `take 2 [1, 2, 3] ~> [1, 2]` (los n primeros)
- `drop 2 [1, 2, 3] ~> [3]` (todo menos los n primeros)
- `elem 3 [1, 2] ~> False` (pertenencia)
- `concat [[1, 2], [3]] ~> [1, 2, 3]`

- `sum [1, 2, 3] ~> 6`
- `product [1, 2, 3] ~> 6`
- `and [True, True, False] ~> False`
- `or [True, True, False] ~> True`
- `minimum [1, 2, 3] ~> 1`
- `maximum [1, 2, 3] ~> 3`
- `zip [1, 2, 3] ['a', 'b', 'c'] ~> [(1, 'a'), (2, 'b'), (3, 'c')]`

Comparación de listas

- Dos listas del mismo tipo pueden compararse si sus elementos pueden compararse.

```
cualEsMenor :: [Int] -> [Int] -> String
cualEsMenor xs xs' = if xs < xs' then "la primera"
                    else if xs == xs' then "iguales"
                    else "la segunda"
```

- `cualEsMenor [1, 2, 3] [1, 2, 4] ~> "la primera"`
- `cualEsMenor [1, 2, 3] [1, 2, 3, 0] ~> "la primera"`
- No necesitamos definir `<` y `==` para nuestras listas.
 - Haskell los deriva del orden de `Int`, comparando lexicográficamente.
 - * Y una lista que es prefijo de otra es menor que ella: `[1, 2, 3] < [1, 2, 3, 0]`.

Rangos

- `[1..4] = [1, 2, 3, 4]`
- `[2, 4..10] = [2, 4, 6, 8, 10]`
 - El paso se infiere de los dos primeros elementos.
- `[10, 9..1] = [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]`
- `[4..] = [4, 5, 6, 7, 8, ...]`
- `[4, 6..] = [4, 6, 8, 10, 12, ...]`
- `[13, 26..24*13] = take 24 [13, 26..]`

Ejercicios

1. Declara y define estas funciones sobre listas:
 - `todos` (igual que `and`)
 - `cremallera` (igual que `zip`: `cremallera [1, 2] [3, 4] ~> [(1, 3), (2, 4)]`)
 - `aplana` (igual que `concat`)
2. Define sin usar recursión explícita la función `mitad` tal que
 - `mitad [1, 2, 3, 4, 5, 6] ~> ([1, 2, 3], [4, 5, 6])`.